

ActInf ModelStream 017.1 What's Wrong with Active Inference and How Lazy Dynamics Is Set to Solve It

Source: [YouTube](#) Playlist: ModelStreams Duration: 1:05:47 | Uploaded:

Unknown Transcript method: auto_caption

Hello, welcome everyone. It's March 21st, 2025 and we are here with a bunch of the developers of RX Inference and Lazy Dynamics. We will be having a presentation followed by a discussion. So looking forward to everyone's comments here on the panel and to anyone watching live looking forward to your questions in the live chat. Albert to you for the presentation and thanks everybody again for joining. Hi hi everyone. Yeah thanks Daniel for uh uh this introduction. Um yeah so uh for everyone my name is Albert. I'm a CEO of lazy dynamics uh which is a spin out from Eindhoven University of Technology and Lazy Dynamics essentially builds uh infrastructure for developing adaptive agents uh such as uh active inference agents and today yeah I'm pleased to to have uh almost full lazy dynamics team here with me. So yeah let's uh let's get started. Um I must say that actually yeah this talk is uh going to be admittedly uh provocative perhaps even a clickbait uh but uh yeah let's just have some fun here and actually I hope that many of active inference um practitioners can uh relate to the pain point I'm about to discuss. So yeah, now I kind of expect that uh yeah uh some people won't like the way we posed the question and uh yeah, I'm hoping um I'm actually not going to to answer in the way that people might anticipate because uh yes, active inference is a um theoretical framework which is very elegant, very powerful um but um if you look at this from a practitioner perspective, uh it leaves much to be desired. So from a framework that claims to be superior to reinforcement learning that promises to revolutionize AI that's supposed to explain actually everything from uh bacterial behavior to human consciousness. Uh what do we uh see in practice, right? What uh uh what what do people who are getting excited actually see in practice? Um well look at this h beautiful theory uh groundbreaking uh or like uh equations and yet uh this is sort of a typical reaction from uh from a developer. So they're literally like running away and I don't think we can actually blame them uh for that because u well we we've somehow managed to take one of the most promising frameworks for intelligent systems and make it so complex that uh even a PhD

student would think twice before implementing it. So, and I just want to draw sort of a parallel and look at what's happening in the rest of AI. Yeah. So, five lines of code, that's all it takes to access the state-of-the-art transformer models. So, if you want to analyze satellite imagery, done. Want to create a conversational agent? Done. Uh, want to synthesize a video or audio? Done. So half a page of the unique code. Uh but meanwhile implementing an active inference agent. Well, let me let me just walk you through that. So what it takes. First you read through hundreds of pages of dense mathematical theory. Then you spend months on implementing some custom algorithms. Then you debug some mysterious convergence issues. And after all that work, what do we have? Well, more often than not, it's going to be a teammate example or like maybe a mountain car. Yeah. I mean, come on, right? Like, what is what is this? You just like you just made me read dozens of pages of heavy math to console me with a moving pixels on the screen, right? So, this is quite a leap between I'm better than reinforcement learning and I can do a teammates better than reinforcement learning. So, I mean, come on. So, we we're like in 2025 now. So the rest of AI is solving like really interesting world problems and like having really interesting demos and we are still celebrating when our agents can turn left or right. So and truth to be told I mean I think my personal experience is that yeah like each time I'm getting into this active inference literature it's uh to put it mildly not always very much reproducible or like half of it is not there's not much to reproducing. So the point here is that this needs to be changed. So uh and not just for convenience but because this complexity is I think actively holding back the entire field. We are basically limiting the active inference u uh to a tiny group of theoretical specialist when it should be in the hands of every AI developer. Um and the saddest part actually of all this that people who used to be passionate about the theory they just give up on it and we just see it all around uh like in the community. So how can we fix that? How can we actually uh make this uh as accessible as transformers? So for starters uh we must recognize what makes active inference agent unique. Yeah. So well according to the theory it's obviously not the inference uh mechanics right because the inference mechanics is same across all agents is just free energy minimization. So what's left what's left for us like this three three lines of code is basically the model and constraints. So this is what um uh this is the only unique part this is the only like this unique three lines of code if you wish um uh which is uh which defines the agent so it's it's should be only about model definition and constraints um and that's exactly what we are trying to do with ax infer uh we developed the framework that we are we are continuously developing this framework that allows you to provide a constraints and run the inference guided by the yeah free energy

minimization and I'll talk about this uh in the next slide a bit more um and what we actually can see is that once you have the framework right or let's say almost right yeah because it's a development project you can streamline the development of these agents by orders of magnitude and u seriously like when started my PhD. I spent few years trying to build an audio counterpart of a teammates demo. So uh that that people can now build in just few hours. So that's kind of the order of magnitude you you should uh think of. And yeah, just to zoom in a little bit on some of the examples. Uh now basically if you want to build a drone that does uh that does planning through inference it would just take about 10 lines of code to define the drone model and then like run inference to get this nice path planning. Um so here you see like uh two 3D drone uh examples. One is flying in the rainy day, another one is in the sunny day and another one is like 2D drone which I'm more excited about because it's actually shows this reactivity aspect of our uh framework. So you could see that we are changing the environment and the drone doesn't hold it continues to sort of reinfer its policies and its states in order to reach this goal no matter the environmental changes. So yeah 10 lines of code. So um if you want to yeah same same applies to locomotion control. So there are two examples one for robotic arm another one for this uh there's a name for this uh robot I for but basically this swinging swinging uh monkey robot right so let's uh let's phrase it this way um so again define your model run inference um if you want to have multi-agent navigation in a dynamic environment with collision avoidance and speed constraints that would take you like 15 lines of not uh thousand just uh 15 that's uh quite quite a boost I would say. So uh same yeah another just uh few demos uh from our um repository again if you want to integrate uh deep learning uh into the uh into our infer it's also possible you can learn the dynamics and like predict the future sequences all in one go. So it's not two separate uh function calls you just define it you just call the inference within the same model. So eight eight yeah here's like about eight lines of code and yeah surely yeah how could we avoid mentioning a better version of teammates. So that's that's something not publicly available though but that's something we've been working on there kind of an agent that needs to um solve a mini grid. So there is an agent that's needs first to locate the key then take the key then figure out that he needs to open the door and then go to the to the to the exit. So um that's something in works. This is the only example which is not publicly available yet uh but it will be soon. So uh and yeah before I go uh into how we are doing that I just want to say also that um uh these examples themselves they are hardly as impressive as reinforcement learning demos or to critique MPC demos. uh but they are better than I would say this stigmatized examples of uh

active inference supremacy. So um yeah um I think that um it's still ongoing uh but it's it's it's going pretty fast. Um and um yeah so how we do it? What's our approach? Um so we do it through essentially four key key innovation. So the the first one is most important is like free energy minimization. It's automated. uh it you uh no manual derivations uh needed for a very large class of models and there are like they're composable but that's basically why we adhere to factor graphs because a factor graphs is a very convenient underlying model representation which allows us to essentially seamlessly combine uh different models into bigger models or the other way around decompose bigger models into smaller models. Um, and the yeah and another thing, yeah, we we of course we adhere to a synchronous data uh such that we can handle and facilitate this real-time adaptation to changing environments. And the really key thing here is reactive message passing technology. Um, and in our opinion, this is the actual key to successful implementation of functional agents or like functional active inference agents. Um and the argument here is that see the world uh the real world I mean it doesn't start and stop at the agent's convenience. So the agent must react adapt act upon env environmental data streams. So if something unexpected happens and like the world always brings some some uh unexpected uh things uh yeah to you and yeah you essentially you you have to adapt. So we must compute our policies. we must compute our posteriors no matter the failure. So that was what you could see with that that little drone demo. Um and so this is what our implementation of um I think so this is something that uh like what the other implementation of active inference aren't getting quite right. uh I think uh and actually yeah to be fair most of the reinforcement learning libraries adhere also to this imperative updates of their policies and so on. Um nevertheless I don't want to give the impression that the current implementation of reactive message passing works as we wish it would. uh it's super fast uh we are like uh beating uh in terms of inference many other frameworks but we also recognize how it can be improved and the truth to be told can be it can be improved we can actually make it faster and better that is something we are working on as well so what's the state of arxenfur today um arx infur is was and will remain the open source library that has more than now 10 years of cumulative development time. We are about 10 active developers from laser dynamics and bias lab. BIOS lab is the laboratory where we spun out from. Uh and of course active inference institute uh who is helping us with the development of the graphical interface with demos like there's really lots of help coming from active inference that we appreciate. Um, so we have over 700 uh 30 stars on GitHub across reactive uh based packages and uh more than 300 monthly downloads. Now I mean that that these are not the craziest number you could

think of but um I think here's why these numbers actually matter. uh they show that when despite all the barriers, despite all the complexity, despite having to learn Julia, despite having to learn probabilistic programming, uh people are still pushing to use active inference. uh they push to use arx infer in general and we are seeing this uh and just just try to imagine like if we are seeing that much um interest uh when it is hard I just want you to imagine uh what would happen when we make it easy and that's kind of exactly what we are building toward so before infer in the early days of my PJ and I think even prior to that uh I had to write quite some code uh actually uh do quite some math. Um now uh with uh bias lab and reactive based community we pushed it to the one page of model definition and like everything else is automated but at laser dynamics we also recognized that even that is too hard. So even even that is uh really not going to uh streamline the development of active inference agents because there is like a huge python community and like uh people want to integrate uh our models with with their models and so forth. So that's what that's where LA dynamics came in. uh so we are actually going to hide the complexity even f further and wrap it uh wrap the models into a nice open available for python rust and so on. Uh so we do recognize that uh model specification is still hard for many users. So in the beginning we will provide models for users and developers. Uh we will start simple and then we will make it more and more uh interesting and complex. So with the vision of that people can actually will be able to submit their models uh host their models and we will just provide the API uh to trigger those. Uh so a bit of uh teaser uh it's not available uh for mass use. It's something we are testing internally. Uh so how that's going to look like? Uh well um so one would basically have to load the model from our pool of RX infer models and then run inference/planning take actions and learn. So we will do it now in this separate API calls for clarity. Um but I mean the mechanics under the hood are the same. So it's just free energy minimization. We just realize that yeah people get a hang of it faster if you just pose it like this. But yeah, it's a work in progress. Uh we plan to release this API in the early Q2. Uh so it won't be as flexible as RX infer but it will expose endpoints and we will make it better over time. Um yeah and this is uh uh the team that has this ambition like this le dynamics uh team but I must say that there are more people behind this now. So there are more people behind uh arx and fur development. uh this is just solely the L dynamics team uh the yeah um but there are more developers as I said outside this le dynamics team so uh yeah just uh yeah I think last few bites for me is that yeah I think that active inference uh needs to move finally from like this beautiful beyond beautiful theory and into practical applications uh we are showing that it's possible and the framework is indeed revolutionary and I think that

now it's time to make it usable. Uh so yeah, thank you for listening and uh here I'd like to stop and actually invite people for questions. Awesome. Thank you. If any of the others here just would like to go around say hello if they'd like any first thoughts and meanwhile anyone in the live chat can write questions. Which order do you want to go? Uh Daniel I can I can start. I I think it's fantastic what uh Albert and uh and the team is uh developing. Um I still believe uh active inference um is the future also to reinforcement learning uh uh problems. Um and uh it's the tool set uh user-friendly tools that's that's lacking. So uh this is an initiative I think that hopefully will uh will get us closer to yeah to to recognition of active inference as a u yeah as the underlying framework for for future intelligent agents. Um and uh yeah, everybody is uh invited to uh to contribute to uh to critique uh to give feedback. So um yeah, we're we're very open uh with in terms of um how we should develop uh towards the future because we don't have all the answers ourselves. Um but um yeah, this is uh this is our push first push to make it as simple as uh as working with transformers. Thank you Mitri. Do you want to add add few words as a core architect of uh Rick and Fur? Um well I I can only add that you was a bit wrong saying that it's not publicly available. Actually the API is almost publicly available. We have released the initial version of it. We just didn't advertise it as much and it doesn't really have all the models that we have. But we we are planning to release the API that Albert has mentioned the ser for free for now. So everyone can experiment and everyone can break it as fast as possible because as soon as we fail we know how to fix it, right? So fail fast approach. We will we will deliver new models, we will test it, we will fail, uh we will improve. Uh and for this we need u we essentially need more people to try it out uh because uh we we have our internal use cases we try it out it it works on our internal cases but there are many other people who also want to try this approach. So we will co-develop the model. So we'll release them for free for now for the free API and uh uh we'll see how it goes. And this is just the tip of the iceberg work right. So I think uh the team of laser dynamics we have already had the pleasure to work with the API. So a year ago we all didn't think it would be possible but especially with a lot of effort from the Mitri we're now able to call RXer from Python. So this should integrate very easily with existing um reinforcement learning toolboxes out there and I've already had the opportunity to play with it. I'm playing with it also before this live stream and I'm really enjoying it so far. So basically merging the infrastructure that's available in Julia with everything that's already available in Python. Um and I think that's just the tip of the iceberg. There's a lot more in store coming up real quick. Um yeah so everyone would want to try it out also just contact us reach out share contact details whatever you want um because I think

we should all adopt this approach maybe just bar one follow-up question there like in integrating through Python calls to RX infer where do you see that plugging in for somebody who who wants to use it amidst the other machine learning methods they're already deploying like what would that look like how would they conceptualize of their pipeline and where these kinds of probabilistic models would fit in I think these probabilistic models themselves they're very important when you are trying to model a process that's like involves uncertainty or this autonom yeah making these autonomous systems so in those regards I think that's for current machine learning frameworks often lack is that usually there's this blackbox neural network. I think personally that having a probabistic take on specific aspects being just in the latent space of a neural network, right? I think that could benefit a lot of enterprises and a lot of products making them more reliable and robust. And because most of these products have like um some back end some machine learning models already in Python, I think that would be amazing to integrate RX refer from Julia into their existing um infrastructure. Yeah, just on on top of that um there is an opportunity that we well not the opportunity there are a few demos people are trying out is basically to connect their uh LLM based agents with uh to yeah to connect them to uh the back end uh of RX infer that's uh that's also something uh feasible and apart from it uh so there are many problems indeed that can be tackled with uh probabilistic programming languages, right? like uh you can there there are many at the table like MMC and Nyro and um yeah people use them as components into their uh bigger like machine learning pipelines and here the well now you basically can uh can do it with RX infer just basically uh without leaving your Python environment you can call uh RX defer to provide you some uncertainty estimates and uh yeah confidence over the answers uh for other of uh parts of your machine learning pipelines. Can also add maybe here a little bit to expand how we can integrate with LLMs because we can imagine that LLM is a sort of is a source of noisy data. For example, we can ask questions to LLM to give some numerical answers. Uh but you should not really trust it. So you may you may think of it as a noisy observation of a real data. So and you can create a model that takes into it into account uh and basically treat this as your observations and then the underlying plistic model will infer the real parameters of the process. So I think uh yeah rix infer can be integrated very naturally in the existing pipeline. So it shouldn't really be or it's not necessary to have just a fur in your pipeline. and can be integrated with like other data collection libraries or the existing machine learning u libraries. So yeah, I think uh but uh it it excels when you really need to create a plan given a noisy set of observation and given the model of the process. Cool. um Ismail if you'd like to

add anything and then otherwise maybe to Frasier and then we'll go to some pre-written questions in the live chat. Yeah, sure. I'd like a couple of uh things to say. Um first of all the uh new uh RX and first server I think will add extraordinary capabilities uh for both Python developers and for Julia developers that are uh struggling find the right platform to develop agents that are not hallucinating. Um yeah so for that big uh many thanks to Dimitry for uh realizing this and uh yeah I would like to invite everyone uh who is keen on learning about machine learning and developing agents or developing models for whatever task they have in mind to uh try out RX infer and uh engage with us uh their problems and uh their experiences whether it's positive or negative that will be a very valuable uh formation for us and that's uh that's mostly it if I can add uh I I see a a little bit like a big focus on Python but Albert didn't really focus on that much but uh the the way we created Ericson first server is with the open API specification that allows to use the same API from uh any language that supports uh open API which is which includes uh basically all languages uh that exist including Julia. So it is possible to use Julia for Rick and first server or Python or JavaScript or Rust or even C and uh if I'm not mistaken even microcontrollers they they do have some sort of a layer between this open API specification. So it really opens doors not just for Python developers but uh also from the browser we can now use our X infer because JavaScript is also supported. So it really opens doors like uh to for experimentation in many fields. Awesome. Brazier, you prepared some excellent comments. So however you'd like to continue, go for it. Cool. Thank you. Um I should just clarify, I'm not a developer on RX and Ferrer. I'm I'm an open source contributor and I'm a PhD student at Glasgow. I'm not part of the core team. So I have I I think Daniel what I'll do is I'll condense those questions I had down into two hopefully just to save a bit of time. The first question is about active inference more generally because this talk was well thank you first of all thank you for the talk. It was it was very enlightening. It was excellent stuff. Thank you Albert. But about active inference the problems with active inference uh you know and how how arx and forest kind of address to uh to to solve them. It seems to my mind basically that we have issues with uh real-time inference. We have issues with robust inference and scalable inference and then even beyond that uh scaling say like the the generative model itself to be able to take care of more sophisticated inference queries. Those kind of three things seem to me to be the the current issues with active inference making it real time robust and reliable. Yeah. I I mean RX infer has effectively been uh kind of the best tool in game in the game for for addressing those concerns because of its its very nature but can anyone speak to let's say perhaps how uh continuing developments of RX are

going to to assist in those three kind of domains and I have a follow-up question about RX and first server so I know it's very broad but u alas I had to ask it m do you want to answer which one yeah yeah I can answer it's a great question so uh right After arix infer server what we plan uh we want we want to change uh and uh refactor the core of fur to make the points that you mentioned uh even easier to do even easier to scale and faster to uh to run inference but but also to make it easier to debug and to contribute for other people because also with the experience with you we we we noticed that it's quite difficult to contribute because the uh the first of all it's written in Julia uh so not a lot of people know Julia but also because it's uh uh it has been it has started as a PhD project and certain parts weren't really designed very well so we will take a step back and we'll uh given all the knowledge that we have now and given all the feedback and given all the goals that we and other people want to achieve we'll redesign the the the entire core of reactivemp uh and this this potentially will open the doors also to run the inference on GPUs for example so that's that's how transformers basically scaled themselves right they they started to use GPUs at all uh at at large scale so we have big plans to to improve uh the scalability even further and make the inference even faster uh and easier to maintain and debug and contribute for other people. Excellent. That sounds like a small, you know, uh, problem. Um, did I cut someone else off there? I sensed another voice. Um, yeah. Well, that was me. I mean, I you didn't really cut me off, but um, yeah, I perhaps I can add to that a bit, but I'm more of a big picture person. And so maybe also for people that are uh just wondering about active inference and are still uh in in other areas of machine learning like reinforcement learning or deep learning, why would they be interested? So um the I think most people agree that the basian inference is is is a correct framework for for dealing with uncertainty. But the big thing is the u is the computational cost and that limits us in scaling this all up, right? Um so people have mentioned uh ideas like well maybe we should do planning as inference. um this is possible with arch and fur and we we're working on stuff that we uh will probably reveal also in the next uh half year where uh I think it will be a nice surprise in how to do planning as inference and then there is a an even more broader idea that uh this idea that you shouldn't do basian inference because of the computational cost which limits the scaling I think it's almost the opposite around by um doing bijian inference You have access to where the uncertainties are and therefore you also have access to when you just should stop computing because you shouldn't you shouldn't compute more accurately than than is needed right when when your data is uh when you have large variance about your data or your variables you don't need very accurate computing so um I think once we're done with basian

inference I mean not just lazy dynamics but the whole community it's going to turn out that it is not that it's going to be because of doing it doing basian inference that we can do actually uh scale up intelligent agents to very large problems. So Be Asian inference will not be uh the the hindrance. It will be a catalyst for scaling up. Um so that's the I think why people should get interested in uh in active inference even if they're still in just researching at this moment. That was it. Excellent. Thank you Bart. I'm very very very keen on the entire methodology here. Just my last question and then I'll I'll open it up to everybody else. So about RX infer server I I myself I played around creating a few active inference agents bonafide agents with RX infer which is is currently difficult because RX infer you know it specifies the inference procedure and automates the way that that happens. The actual design of an agent is is kind of another thing that's you know the inference procedure is one part but at the moment it has I I and anybody else making an agent has to do everything kind of ad hoc. we have to specify well if an action happens here that affects the environment in some specific way. It's all very ad hoc and so on. So my understanding is that the the agent API as part of RX and first server is going to perhaps uh ameliate those issues or or standardize the way in which we talk about how an action influences the environment and vice versa. is is that uh let's say a big focus of the API or is that just now a part of the API in the sense that we can now do much broader things like have endpoints in Python and so on. So that's kind of a larger uh concern than just this API. any yeah I think it's yeah sure I I think Mitri would definitely add upon that just from my side that's that's uh there is actually ongoing discussion that I forgot to mention uh like in our uh GitHub repository about the standardizing API I think one of the things which we are also striving with our uh this RX first server and generally like with streamlining this development of agent is actually to standardize how the API should look like that There should be a certain function that should be triggered uh in uh well in the reactive way or in the loop if you prefer imperative style coding but there should be like really uh sort of we want to impose one way or few ways to actually doing that. So yeah another thing is just want to welcome everyone to to this discussion on um uh yeah Arix infur repository where uh you Fraser and like and other people from active inference institute also participated but the goal of course is uh the end goal is sort of to to bring in the key components which are needed to yeah run agents uh in the way we run transformers uh I mean low code Um, okay. I see. Thank you. Well, Daniel, I'll hand it back to you. Maybe we can look at some questions in the live chat. I'd just like to say thank you very much. Uh, this is an incredible project. I'm very enthusiastic about the the future. Thank you. Awesome. I'll read some questions in the live chat and then we'll see if we have

time for some of the more pre-is popping up. Okay. So, two questions on computational resources. Indian Goat wrote, "Can you scale to large data sets?" And Jeff Beck wrote, "I would have thought that memory costs would have been the critical bottleneck for scaling care to comment." And so to that, I'll just sort of also add a little bit of my own computational resourcing just how do you find that storage, memory, and CPU cost scale? Is it possible to do static analysis? How do you find that resourcing scales in comparison with alternative methods? Yeah, I can start and maybe you can uh roll a bit on this VQVA for example. Can we scale further? But the when we talk about scaling uh we can look at it from two perspectives. The first is the amount of variables we have in the model, right? So we have a million of variables but all of them would say univariate variables. So for this kind of scaling fur already does a pretty good job. So because we use factor graphs and if you have a sparse connectivity it's actually it's only limited indeed by memory in your computer. Uh and I I uh personally I run inference for 2 million univariate variables and it has been done under like 5 minutes or so. it was a state space model. So it it was it was pretty easy. But another aspect of scalability is indeed the the dimension of each individual variable. So for example all the the multivaried and this is indeed this is still a hard problem. So if if you have a very large state vector then indeed you need to propagate their coariances if they're gaussian and this might be tough for which uh now we prefer to use uh mikva uh basically we compress the state into a smaller dimensional state. Albert can talk a little bit more about that. uh or or for example we use techniques uh similar to mean field but when you approximate the coariance matrix with a diagonal matrix which also reduces the memory requirements um the there are many many many many techniques that we can use the thing is that infer doesn't really choose it for you uh it's more like a it's more like a playground for a user for now where they and choose different approximation methods or different uh techniques. But Arixford doesn't restrict you here. You can even try your own uh approximation techniques. But that would require you to write your own message passing rules which we again we we noticed that many users uh uh they cannot do that because it's it's hard uh it's hard and it requires a lot of knowledge on how to do that. Yeah. Ju just to add on on that about this uh what's okay what was the question from Jeff who was like about this memory cost um um yeah so I'm going to answer that in in three parts sort of so the the first one is to use uh sorry to say that non-basian techniques right so or not entirely Beijian so and that's something like actually active inference community should have been doing like uh earlier I think uh so basically what you could do to scale is what Mitri said you can use u semi-basian approaches like vation autoencoders to compress to basically

cast your high dimensional uh uh vector representation into a low dimensional vector right so that that's the basic thing you can actually think of uh you can uh do it in also on multiple levels it doesn't have to be only in the likelihoods right you can do it in the state so I mean as as as you move in your uh in the hier in the hierarchy of your model you can embed this var encoders one after another uh so uh that's one thing uh and another thing is the downside here is that you require pre-training right so there is some pre-training involved uh not cool uh sometimes sometimes it's okay so uh another way of tackling that is to actually use hierarchical model representation. So uh you can encode your state see in as a sort of hierarchical multiscale uh model uh right so you don't have to um uh when you have a grid world for instance you don't have to uh yeah basically model the uh all these uh uh components of your grid into one state right so like you will end up uh Yeah. So it doesn't scale well. So and we know a few frameworks that adhere to that and well I think it will not progress much. But if you actually manage to uh have sort of multiscale representation of of this grid world, right? So you can have sort of like a location in terms of u you can break it down, right? So first you can have a a big grid world where which you can divide by four elements and then you can zoom in zoom in zoom in. So basically you would have sort of this granular representation of your of your state. So and this scales pretty well. Um and another thing is like well it's kind of what Bert uh touched upon right is this uh uh vision uh going forward um is is indeed like to use this reactive approach to well uh to solve this scaling once and for and and uh forever right so it's basically the the idea here is not to spend uh much computational resources just basically spend as much as you can So uh but yeah that that's that's something work in progress. Um yeah and uh I know if B you want to add something on top of it uh about this computational cost I think you have uh something well yeah I mean we we just discussed that right? I mean I think ultimately uh a basian framework will help computational costs because you have access to how much precision you want for your computations and you know that means you can stop early. Uh you can the the nice thing about the message passing process is reactive message passing is that you can interrupt it at any time and take the result as it is and if you wait longer you get more accurate result but if you stop early then you have a less accurate result but it may be okay and you you know in basian framework you have access to how accurate you you want to compute um at the higher levels in these uh uh in these high article models you don't need a lot accuracy, right? Um, if I want to park my car, I'm not going to worry about the the centimeters of uh where I actually park. I just want to find a parking spot, right? So that's I'm talking about the tens 20s of

meters, right? So um um it's so it's it's partly in the architecture and it's partly of doing a smart uh inference in the end. Yeah. And two two more things here. I just realized that uh well you it's so I I've mentioned that yeah you can use nonbasian approaches to train your uh say vageal encoder but actually now there are like ways of training your uh neural networks in online manner also. So it's it's uh not inherently like prohibitive like to actually uh do learning and inference in the goal there. Uh yeah there are some developments in in in that part of uh of that world as well and I think uh the last piece here is that hardware for Beijing inference is also emerging. So it's it's something that people are also working on. uh it's uh it's not at mass scale yet but uh there will be hardware breakthroughs I think that would be specifically targeting this uh um applications for of of Beijing inference if if there are no other comments or questions just now I would like to follow up on this issue of hierarchical modeling maybe maybe we've exhausted it just now but it it is it appears to me uh as it does to many people that this issue of hierarch hierarchy is going to be very important for just active inference going forward. Um you know that this is very much how biological organisms are arranged. They they have characteristic units that deal with their own time scales and they they're connected to some other level above and level below. What's um I know that in ARFO you can create bonafide subm models uh within other at model blocks and so on but is there uh I'm interested in the sort of landscape of hierarchical modeling for RX and for in particular are there are there things that you guys would like to highlight or um emphasize in that regard for um for users like me I I think Dmitri maybe you can just tease a bit about this idea of multi So multiscale or hierarchical models are uh uh they're available right now but uh what's not available out of the box is basically as you said uh different updating schemes for different levels. So that's something uh mitrior that that's what I mentioned about uh rewriting the core of arson fur to also support this feature more naturally. So I can imagine indeed that we have a really huge model with big hierarchies but we don't really want to update um deep hierarchy as often as we update for example the layer that is connected directly to our observations um because yeah as mentioned for for certain tasks we we don't really need a lot of accuracy so we can uh we can sacrifice it there and do less accurate predictions but in the current implementation It's a bit hard to do. It's like it requires a lot of hacks. So we we tried it once. It it in principle it's possible to do but it's like um user unfriendly. So that's also one of the directions that we want to improve when you're writing for tools to better support hierarchal models to to make it easier to initialize hierarchical model to make it easier to support different update rates for for different hierarchies. Awesome. I'll ask a few more questions in

the chat. So, Michael Lenin asks, "The difficulty of implementation is a great provocation point. What are what have been use cases where you have seen some of the greatest benefits from your approach and what have been your most surprising findings?" Uh, great question. I think uh if what what what are the grades? I look so I think technically if you just go to um so what was most surprising I think it's just the fact that uh you use the same technique same inference uh approach same back same back end to tackle all sorts of different problems like here uh there's many problems on control of course but like it goes way beyond that right so there are like some more classical examples on uh auto latent photoregressive models uh gouching processes uh sensor fusion handling missing data. So I think the benefit of our approach in particular like first of all is that it it works close to real time and certain applications is definitely real time not everywhere. Uh but uh yeah, I think just the uh like the the the fact that it's so versatile in terms of different domains uh while uh kind of connecting all of them under the same uh umbrella principle, right? So that's that's I think uh the most powerful finding um of Arix infer in particular. And yeah so it's also um a reason why um just generally this approach is not super popular I think in engineering communities right act of inference hasn't is not a big name in in the engineering communities because as engineers and we are an engineering group we are used to um you start with a problem and then you go to the literature you find 10 uh possible solutions and you pick one and you you change it a bit and now you have 11 solutions approaches. Um this is entirely the other way around. For every whatever the problem is, we have the same approach. We build a model, we do inference. It's one solution approach to any model any any problem right so it's completely the opposite way of thinking of problem solving for an engineer. And it's very hard I noticed it with with the students in my class. It's very hard to make that switch from uh one problem has lots of possible solutions to we have uh one solution method for all the problems. Uh that is uh that's very hard and uh um but but it works right. I mean it's uh yeah I think this is the right way. uh but it's it's it's going to be difficult to make that switch in uh in the engineering communities. Awesome. Just again to footnote and include some of these great questions and and welcome people into this continuing conversation and open source development. So a few more. Okay. Urban ABSAC wrote, "Does RX infer and if so to what extent support structure and parameter learning to train and improve models automatically?" Okay. So yeah, that's a good question. Um I wouldn't say that there is automatic way of doing that. You can there are I think examples on structured learning to some extent and and look so one thing is that people uh uh so sometimes people mean different things by referring to structured learning if we're

talking about uh finding this kind of so sort of this discrete program search to automatically discover what is the right graph for your particular problem. Uh it's I think there's a lot of ongoing research. uh it's not set in stone how to do it uh and uh I think we have good uh wouldn't say intuition we have good understanding on how to prune the models based on this uh well essentially obviously bijian model evidence approaches so we can definitely use that within arx infer but how do you extend the model how you grow sort of your graph I think this is still an open question an interesting one um And as for uh another way of looking at actually uh uh structure learning right or uh you can think of uh nonparametric uh methods including gaussian processes for instance and those are also going to be integrated in rix infur I think yeah hopefully this year but the idea is that yeah you can just basically discover uh this distribution over your u uh over the functions uh that drive you that explain the dynamics of the environment. So uh there's we are certainly working on that from different angles and there is also bias lab that Bert is leading that also tackles some of these issues. Uh as for parameter learning yeah I think this is sort of uh I wouldn't say solved right not not nothing is like 100% solved but I think that's something we are uh good at. There is no problem with uh parameter learning and and as I said pruning that's available. Yeah. Awesome. Yeah. And batch and streaming inference for parameter learning. Okay. Andrew Pasha wrote, "Thank you for the updates. How are you currently handling multi- agent environment loops iteratively, eg creating n agents at the top of the script and using loops or functions for creating and storing them as opposed to hard- coding each individual agents. So what kind of design patterns have you been working with or do you think are relevant for multi- agent situations? Yeah, multi- a yeah thanks thanks uh for the question. Multi-agent is uh of course like is like a pinnacle sort of all these inference problems. I think uh there well you one way is like indeed to hack it through your loop. Uh another way is um actually like what you what you see here in this multi- agent path planning is that it's a sort of a super agent. So there's kind of this sort of a meta agent that encapsulates the models of all other agents and then they share this information. Now the problem is that it doesn't scale well I would say because well like four agents, five agents, 10 agents fine but then like uh you you you you get right you get it. So I think uh one of the ways I mean Mitra probably can can also add upon that but uh what you could do like as for the design pattern is uh you run your agent uh for instance like speaking of this uh path planning. So you run your agent into the you let it go into your environment and you only interact when you are in the proximity of other agent. So there is not need to share any information if you are far away from each other and like once you are getting closer you

can start sharing the information you can sort of uh fuse the models figure out the conflict and then just like depart. So that's that's one of the things u I can suggest but uh Andrew's question there is no uh we we we didn't think of how we we didn't let me put it this way we didn't standardize the API call for multi- agents so not yet all right great and one more question from live chat Jeff Beck wrote my question is can the parameters of the flows and the VAE be learned in RX and FUR or are they imported? If it is learned, is that by traditional gradient descent? Well, I think this is if it is still here, I think he can elaborate on that. But uh you you can yeah, maybe I can talk a bit on that. So it's uh possible to update um weights or biases of the neural network that is parameterizing your VAE or your flow within uh uh RX infer. However, if you want to use deep and wide networks then you need uh the GPU architecture that the Datri uh was referring to. So uh for small to size uh neural networks v you can use RX infer to tune the weights but uh for larger scales in its current have used uh weights trained outside of the model and then use these parameters uh for prediction purposes. But it is certainly possible to incorporate the training of uh neural networks within the model macros. Awesome. Okay. In sort of our closing round here, one of the pre-written questions was what are the upcoming milestones of lazy dynamics and what is your vision? So just on that though or however else you want to give some last thoughts uh if each of the developers would just want to give any last thoughts what are they looking forward to this year where might we be by the midpoint of the year by whatever milestone or timeline you want to point to and that'll be a great closing route. So, how about Albert and then continuing on with the developers? [Music] uh well uh bring sorry that it was this approach in particular to enterprise use so we see a lot of opportunities I I'm sorry I made just an audio mistake so people might have lost audio for a few seconds just could you just start again start again yes sorry from where from just just all over. Yeah. Sorry about that. Yeah. So, uh two two things then uh like first very first thing is that I I definitely want to invite uh so lazy dynamics is going to support the uh open source development of RX infer. So that's our priority on the development side. Uh we are actually going to we are planning to run a sort of not a hackathon but the uh development uh event um core development event uh probably this summer uh and uh yeah so I just want to invite more developers to help and testers to help with um um agent development uh to help with demos uh even though you are not uh intoxic and furry uh sorry active inference yet right so everyone is welcome and um one core thing for us I think for this year uh from the development perspective is of course to uh bring uh to to put Arix to the state where it becomes easier to contribute easier to contribute for uh to the sort of to the engine not just on

the kind of this demo side but also to improve the engine to make it faster, more scalable. So that's something we are uh yeah looking forward and I want everyone to uh invite uh to that venture. Um and then I think from less dynamics perspective uh specifically yeah we're uh inviting the partners and like forward thinking uh um people individuals uh right who just uh want to uh connect and want to join us in this uh uh adventure of uh building this uh well very scalable Beijing inference uh solutions. Uh I think there's a lot of opportunities and uh possibilities uh for um education and enterprise uh and yeah you name it. So yeah that's that's uh some closing words from me. Okay. So from from the research side I think uh a big issue for active influence is um the computation of expected free energy and computation of policies but minimizing expected free energy and u um I think we're making some nice progress there and we I think we will also release that this year somewhere at the conference and in the package um on how to do this with with arx infer in an efficient way of From the development side, uh is of course as has been mentioned already, we are uh releasing a risk for server for now for everyone for to use to play and uh as soon as it's uh finished. It's never finished, but uh we will of course improve it. But this as soon as it has some stable form uh we will uh rewrite the core of RX and then we also have other uh pending topics like integrating gaussian process very easily making it very easy to use mixture models making it easy to introduce hierarchies in the model. So yes, we have a lot of plans for the development. We also have plans uh to improve the visualizations, right? Since Fraser helped a lot with that and we have we have plans to improve that even further. I think all the improvements and milestones were already mentioned. Um so I'm not going to add more to that. I think we're already have quite a big list. Nonetheless, there are a lot of surprises coming up. So uh definitely stay tuned. I think we already gave a good impression of what direction we'll be working towards, but along the way there will always be more cool stuff coming. So definitely follow us everywhere and uh I think that's already uh yeah gives a good impression of what we'll be up to. Awesome. Um yeah it's hard to add on top of what Albert Bart all the team said but uh as a personal uh interest I'm very uh interested in stoastic differential equations and uh continuous time systems and um we numerically approximate the systems uh identify their parameters and so on and uh this is one of the research that uh we are doing also at RX per and uh how we can uh integrate them. So that's a big research plan for myself and uh yeah it's uh too much on the plate and uh we could uh enjoy contribution from everyone to this uh big plate of ideas. Uh Razer where does that take you? I just want to say I'm extraordinarily excited about um continuing to contribute with uh with you guys and develop the package further, maybe even good, you

know, help with some foray into the the refactor of the package. I know that can be intimidating though, so we'll see. So, yeah, I look forward to to working with you guys in the future. I look forward to it all. Thank you. Thanks, Fasia. Awesome. Thank you again for joining and just another chapter in in the journey and in the learning and in the application of active inference. So till next time. Thank you all. Thanks Daniel. Thanks all. Bye-bye. Bye. Byebye.